

ARTIFICIAL INTELLIGENCE AGENTS: LABORATORY EXERCISE

LAB 2 — CONSTRUCTING A GOAL-BASED AGENT USING A LARGE LANGUAGE MODEL

1. OBJECTIVE

The objective of this laboratory is to construct a goal-based intelligent agent for a warehouse navigation problem using a Large Language Model (LLM) as a software engineering assistant.

The exercise focuses on understanding goal-based agents, designing the agent and its environment, generating Python code with an LLM, executing and testing the generated program, and evaluating the selected search algorithm.

2. PROBLEM DESCRIPTION

The environment is a two-dimensional warehouse represented as a grid. The vehicle starts at the cell marked S and must reach the cell marked G.

Cells marked # are obstacles and cannot be crossed, while . represents a free cell. The vehicle can move one grid square at a time in four directions:

• Up • Down • Left • Right

The warehouse map used in the experiment was:

```
#####  
#S....#.....G#  
#.##....#####..  
#....##.....#  
#.#####.###.#.###..  
#.....#.....#  
#####
```

The objective is to find a collision-free path from S to G.

3. TASK 1 — PROBLEM ANALYSIS

Environment: A 2D warehouse grid containing free cells, obstacles, a start position, and a goal position.

Goal: Find a collision-free path from S to G.

Actions: Move Up, Down, Left, or Right by one grid square.

Information maintained: The agent maintains its current position, warehouse layout, goal position, obstacle locations, and valid neighboring cells.

Agent type: The agent is goal-based because actions are selected with respect to an explicit objective: reaching G from S.

A simple reflex agent would instead select actions mainly based on the current situation without explicitly searching for a sequence leading to a specified goal.

For a larger warehouse, the same general approach can be used, but the number of possible states and paths increases. Efficient search therefore becomes more important because exploration can require more time and memory.

4. TASK 2 — AGENT DESIGN

The agent represents the warehouse as a two-dimensional grid. Its current state is the row and column of the vehicle, while the goal state is the position of G.

The decision-making component searches through possible movements while avoiding obstacles.

Agent Components: Specification

Environment: Warehouse 2D grid

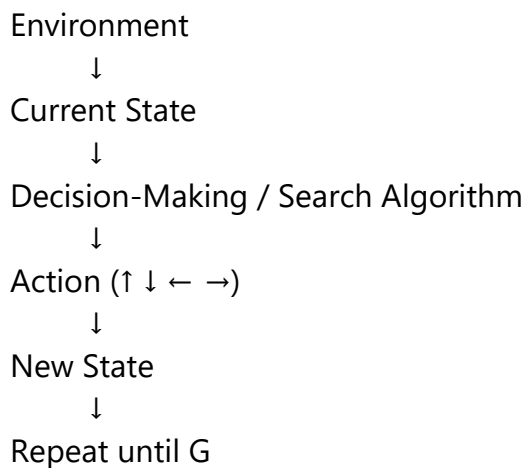
Current State: Row and column of the vehicle

Goal: Reach G from S without crossing obstacles

Available Actions: Up, Down, Left, Right

Decision-Making Component: A search algorithm that explores movements and constructs a collision-free path.

AGENT BLOCK DIAGRAM:



5. TASK 3 — LLM-ASSISTED IMPLEMENTATION

A detailed specification was provided to an LLM rather than using a vague request to write a program. The specification described the warehouse map, allowed actions, collision constraints, expected output, and the requirement to explain the selected search algorithm.

PROMPT USED:

Write a well-documented Python program implementing a goal-based agent for the following warehouse navigation problem.

Warehouse map:

```
#####  
#S....#.....G#  
#.##....#####..  
#....##.....#  
#.#####.##.###..  
#.....#.....#  
#####
```

Where :

S is the starting position.

G is the goal position.

represents an obstacle that cannot be crossed.

. represents a free cell.

The vehicle can move one grid square at a time in four directions: Up, Down, Left, and Right.

Requirements:

1. Represent the warehouse as a two-dimensional grid.
2. Locate S and G from the grid.
3. Determine a collision-free path from S to G.
4. The agent must not move outside the grid or through obstacles.
5. Print the path found as a sequence of positions or moves.
6. Print a suitable message if no path exists.
7. Explain the search algorithm used and why it is appropriate for this goal-based navigation problem.
8. Keep the implementation simple and suitable for a laboratory exercise.
9. Include enough comments to make the agent's operation clear.
10. Do not use external libraries beyond Python's standard library.

6. GENERATED PROGRAM AND ALGORITHM

The first LLM-generated implementation used Breadth-First Search (BFS).

The program:

- Represented the warehouse as a 2D grid.
- Located the start and goal positions.
- Checked whether movements were valid.
- Maintained a set of visited positions.
- Stored parent information for path reconstruction.
- Searched for a path from S to G.
- Reconstructed the path once the goal was reached.
- Displayed the warehouse with the discovered path marked using *.

WHY BFS WAS USED

Breadth-First Search explores reachable positions level by level using a queue. In this problem, every movement has the same cost of one grid square. Therefore, BFS is appropriate because it finds a shortest path in terms of the number of moves for this unweighted grid.

7. EXECUTION RESULTS

The first generated program was executed without modification.

The observed results were:

Start position: (1, 1)
Goal position: (1, 19)
Path found: Yes
Number of moves: 20
Obstacles crossed: No
Search algorithm: Breadth-First Search (BFS)

WAREHOUSE WITH DISCOVERED PATH:

```
#####  
#S***.#####G#  
#.#####.##  
#.....#  
#.#####.###.#.###.##  
#.....#  
#####
```

PATH GENERATED:

(1,1) → (1,2) → (1,3) → (1,4) → (2,4) → (2,5) → (2,6) → (2,7) → (1,7) → (1,8) →
(1,9) → (1,10) → (1,11) → (1,12) → (1,13) → (1,14) → (1,15) → (1,16) → (1,17) →
(1,18) → (1,19)

The path contains 21 positions, corresponding to 20 movements.

8. INDEPENDENT PATH VALIDATION

The generated path was not accepted solely based on the LLM's output. A separate validation function was used to test the path.

The validator checked that:

- The path starts at S.
- The path ends at G.
- Every position is inside the warehouse.
- No position in the path is an obstacle.
- Consecutive positions are adjacent.
- Every movement is exactly one grid square.

The validation produced:

Path Validation: Path is valid.

The validation confirmed that the generated solution satisfies the navigation constraints of the specified warehouse.

9. EVALUATION OF THE LLM-GENERATED PROGRAM

The first LLM-generated implementation worked correctly without requiring a second prompting iteration.

The generated program successfully:

- Identified the starting and goal positions.

- Searched for a collision-free path.
- Avoided the obstacles.
- Reached the specified goal.
- Reported the path and number of moves.
- Displayed the discovered path on the warehouse grid.

The independent validation further confirmed that the generated path was valid.

This demonstrates the importance of testing LLM-generated software rather than assuming that generated code is correct simply because it appears reasonable.

10. **CONCLUSION**

This laboratory demonstrated the construction of a goal-based agent for a warehouse navigation problem using an LLM as a software engineering assistant.

The warehouse was modeled as a two-dimensional grid, with the vehicle's position representing the current state and G representing the goal. The agent used Breadth-First Search to explore possible movements and determine a collision-free path.

The first LLM-generated implementation successfully found a 20-move path from S to G. An independent validation function confirmed that the path remained within the warehouse, avoided obstacles, and consisted only of valid movements.

The experiment also demonstrated that a clear and detailed problem specification can help an LLM generate a useful implementation, while execution and independent testing are necessary to verify the correctness of the generated software.